PROYECTO FINAL
Optimización de rutas de reparto con algoritmos genéticos

UNIVERSIDAD POLITECNICA DE ATLAUTLA

Asignatura: Inteligencia artificial.

Docente: Ing. Garcia Lara Javier.

Integrantes del equipo: CAstro Soriano Ronaldo Malcom

Grupo / sección: 171

Actividad 2.6 · Algoritmos genéticos · Implementación en Python con DEAP

Fecha de entrega: 26 de octubre de 2026

Caso empresarial simulado: Distribuidora Horizonte. Un depósito, doce clientes, una ruta de reparto cerrada.

Estado de validación: implementación DEAP incluida; ejecución DEAP pendiente en el entorno del estudiante. Resultados experimentales incluidos: backend independiente de referencia, ejecutado y contrastado con una solución exacta. Esta distinción se mantiene en tablas, figuras y conclusiones.

IA: Perplexity | Prompt: actividad 2.6 + instrucciones adjuntas | Uso: código y redacción | Verificación: reference; DEAP pendiente.

Resumen ejecutivo

Se desarrolla un sistema reproducible para decidir el orden de visita de doce clientes de una distribuidora ficticia. El objetivo es minimizar la distancia total desde un depósito, visitar a cada cliente una sola vez y volver al origen. El modelo corresponde al problema del agente viajero (TSP), sin capacidad ni ventanas de tiempo. Las coordenadas y la operación empresarial son sintéticas: no se atribuyen ahorros observados a una empresa real.

El algoritmo utiliza cromosomas de permutación, una población de 200 individuos, selección por torneo de tres participantes, cruce ordenado OX, mutación por intercambio de índices y dos individuos élite. Se ejecutan exactamente 30 generaciones después de la población inicial y se planean 30 corridas con semillas 0–29. El programa principal implementa estos elementos con DEAP; se incluye además un backend de referencia para auditoría independiente.

En el backend de referencia ejecutado, la mejor ruta fue de 69.706205 km, coincidente con el óptimo exacto para los datos suministrados; el promedio de las 30 corridas fue 71.193627 km y 6 corridas alcanzaron el óptimo. Estos números no certifican la ejecución de DEAP. La replicación incluye instrucciones para completar esa validación y actualizar el reporte antes de la entrega definitiva.

1. Tema, alcance y objetivos

Tema: algoritmos genéticos aplicados a un problema de optimización combinatoria con interpretación empresarial. Sección: 2.6. Un algoritmo genético representa soluciones como individuos y construye nuevas poblaciones mediante selección y variación; DEAP permite expresar explícitamente estas estructuras y operadores (Fortin et al., 2012).

Objetivo general: implementar en Python/DEAP un algoritmo genético que reduzca la distancia de una ruta de reparto y permita auditar la calidad de las soluciones.

Objetivos específicos: (1) formular el TSP; (2) codificar rutas válidas sin duplicados; (3) ejecutar 30 generaciones; (4) registrar convergencia y diversidad; (5) comparar con un orden inicial, vecino más cercano, búsqueda aleatoria y un óptimo exacto; (6) documentar pruebas y replicación; (7) preparar la publicación pública de código y reporte.

2. Planteamiento del problema empresarial

La Distribuidora Horizonte prepara una ruta diaria a partir del orden en que se capturan los pedidos. Ese orden es la línea base del caso y no una medición empresarial. El sistema propone cambiar únicamente la secuencia de visitas: no decide cantidades de producto, flota, turnos ni ubicación del depósito.

Supuestos: un vehículo con capacidad suficiente para todos los pedidos; todos los clientes disponibles; salida y llegada al nodo 0; distancias euclidianas simétricas; igual importancia de cada entrega; ausencia de tráfico, calles de sentido único, peajes y tiempos de servicio. Las coordenadas se expresan en kilómetros en un plano local ficticio, no en latitud y longitud. Por ello la distancia es una aproximación geométrica, no una ruta vial utilizable directamente.

El caso se mantiene en doce clientes para poder certificar el óptimo mediante programación dinámica y contrastarlo con la heurística. La utilidad didáctica no consiste en probar superioridad del AG frente al método exacto en una instancia pequeña, sino en demostrar representación, operadores, experimentación y verificación.

3. Datos del problema y trazabilidad

Fuente: elaboración sintética para esta actividad. No se usaron bases de clientes ni información personal. El archivo clientes.csv es la fuente única; la matriz se calcula a partir de él sin redondear los valores usados en la optimización. Los decimales mostrados en el reporte se redondean solo para lectura.

ID	Punto	x (km)	y (km)
0	Depósito	10	10
1	Cliente 1	2	3
2	Cliente 2	4	17
3	Cliente 3	16	4
4	Cliente 4	18	16
5	Cliente 5	3	11
6	Cliente 6	11	2
7	Cliente 7	8	18
8	Cliente 8	17	10
9	Cliente 9	6	6
10	Cliente 10	13	15
11	Cliente 11	12	8
12	Cliente 12	7	13

Orden inicial: 0 → 1 → 2 → 3 → 4 → 5 → 6 → 7 → 8 → 9 → 10 → 11 → 12 → 0. Su longitud es 152.294241 km. Cada arista incluye el trayecto entre dos visitas y se contabiliza también el regreso al depósito.

4. Formulación matemática

Sea $V = \{0, 1, \ldots, 12\}$, donde 0 es el depósito, y sea $d(i,j) = \sqrt{[(x_i - x_j)^2 + (y_i - y_j)^2]}$. Una solución es una permutación $\pi = (\pi_1, \ldots, \pi_{12})$ de los doce clientes. La función objetivo es:

Minimizar $L(\pi) = d(0, \pi_1) + \Sigma[k=1,\ldots,11]\ d(\pi_k, \pi_{k+1}) + d(\pi_{12}, 0)$.

Restricciones: cada cliente aparece exactamente una vez; no se incluye el depósito entre los genes; la ruta empieza y termina en el depósito. La representación por permutación impone directamente estas restricciones y evita subciclos desconectados. No se usa una suma de aristas binarias sin restricciones de conectividad.

En el programa, los genes son índices 0–11, que se traducen a los ID de clientes 1–12 mediante ID = gen + 1. Esta distinción es necesaria porque cxOrdered espera índices dentro del rango de la longitud del individuo (DEAP Project, s. f.-a). El depósito permanece fuera del cromosoma.

Ejemplo: genes [10, 5, 2, 7, 3, 9, 6, 1, 11, 4, 0, 8] representan la ruta 0 → 11 → 6 → 3 → 8 → 4 → 10 → 7 → 2 → 12 → 5 → 1 → 9 → 0.

El fitness es la propia distancia L, devuelta como tupla de un componente: (L,). DEAP usa weights=(-1.0,) para indicar minimización. No se usa 1/L ni se mezcla maximización con minimización. La evaluación debe retornar una tupla incluso con un solo objetivo (DEAP Project, s. f.-b).

5. Diseño del algoritmo genético

Elemento	Configuración y justificación

Población	200 permutaciones aleatorias válidas; sin sembrar la solución exacta.
Generaciones	30 transiciones; registro inicial g=0 y registros g=1…30.
Selección	Torneo de tamaño 3; presión selectiva moderada.
Cruce	OX / tools.cxOrdered; probabilidad 0.85 por pareja.
Mutación	tools.mutShuffleIndexes; probabilidad 0.25 por individuo.
Probabilidad por gen	indpb=0.08 dentro de la mutación; no es igual a mutpb.
Elitismo	2 mejores copiados; 198 descendientes por generación.
Repeticiones	30 semillas independientes, de 0 a 29.
Parada	Exactamente 30 generaciones, sin parada anticipada.
Reevaluación	Todos los descendientes se reevalúan después de la variación.

Los parámetros son decisiones de diseño fijadas antes del experimento; no se afirma que sean óptimos ni que provengan de una búsqueda exhaustiva de hiperparámetros. El cruce ordenado conserva una subsecuencia y completa con los genes faltantes en un orden relativo, evitando clientes repetidos. La mutación intercambia posiciones sin introducir clientes nuevos ni eliminar visitas (DEAP Project, s. f.-a).

La selección devuelve referencias, no copias independientes. Por ello cada progenitor seleccionado se clona antes del cruce y la mutación. En el backend DEAP se borra el fitness previo y se asigna uno recalculado. Estas medidas evitan modificar a los padres o conservar una aptitud correspondiente a una ruta antigua (DEAP Project, s. f.-b).

La élite se incorpora explícitamente a la siguiente población; almacenar un mejor histórico no equivaldría por sí solo a elitismo. La mejor distancia poblacional debe ser no creciente. La media, en cambio, puede fluctuar porque se introducen descendientes de calidad diversa.

6. Procedimiento y arquitectura de software

Flujo: leer CSV → validar identificadores y coordenadas → calcular matriz → obtener comparadores → inicializar población → evaluar generación 0 → repetir selección, clonación, cruce, mutación, validación y elitismo durante 30 generaciones → registrar métricas → repetir para 30 semillas → exportar CSV, JSON y gráficas.

Módulos funcionales: read_data valida los datos; matrix genera distancias; distance cierra la ruta; valid comprueba la permutación; run_ga ejecuta el AG; exact resuelve el control exacto; nearest obtiene vecino más cercano; main coordina las corridas y exportaciones. El argumento --backend deap es la opción predeterminada y requiere la biblioteca; si falta, el programa termina con una instrucción de instalación y no cambia silenciosamente de motor.

El backend reference contiene operadores independientes con el mismo diseño conceptual. Su salida se etiqueta y se guarda separadamente. No se presupone equivalencia bit a bit entre backends ni que una misma semilla deba producir idéntica trayectoria en ambos; solo se exige reproducibilidad dentro del mismo backend y entorno.

7. Diseño experimental y criterios de evaluación

Se comparan cinco referencias: orden de captura determinista; vecino más cercano desde el depósito, con desempate por ID; búsqueda aleatoria independiente; AG; solución exacta. La búsqueda aleatoria prueba 6,140 candidatos por semilla: 200 iniciales más 198 nuevos candidatos por cada una de las 30 generaciones del AG. Se iguala el número lógico de

candidatos puntuados, no el tiempo ni el número total de llamadas de distancia del programa, que recalcula valores para registrar métricas.

El óptimo se calcula mediante programación dinámica sobre subconjuntos (Bellman–Held–Karp). Para un subconjunto S de clientes y un último cliente j, $C(S,j) = \min[k \in S\setminus\{j\}] \{C(S\setminus\{j\},k) + d(k,j)\}$, con $C(\{j\},j)=d(0,j)$. Finalmente $L^* = \min[j] \{C(T,j)+d(j,0)\}$, donde T contiene a todos los clientes. Esta recurrencia explora sistemáticamente los estados y certifica el óptimo de esta instancia; su complejidad temporal es $O(n^2 \cdot 2^n)$, por lo que no se propone como solución escalable para grandes conjuntos (véase la discusión de la recurrencia en cstheory.stackexchange.com, s. f.).

Métricas: longitud final por corrida; mejor, promedio, desviación estándar muestral y peor resultado; brecha = $100\cdot(L/L^* - 1)$; porcentaje de reducción = $100\cdot(L_{base} - L_{AG})/L_{base}$; diversidad = número de permutaciones distintas / 200. Esta diversidad mide secuencias, no clases de rutas equivalentes por inversión. Se registra el historial completo, incluida la media poblacional.

La corrida mostrada en la figura individual es la de menor longitud entre las 30, con desempate por la primera semilla. Se declara esta selección para no confundirla con una corrida representativa promedio. No se realizaron pruebas de significancia ni mediciones controladas de velocidad.

8. Resultados ejecutados: backend de referencia

Todos los resultados de esta sección proceden de la ejecución real del backend reference en Python 3.12.8. DEAP no estaba instalado: su ejecución queda pendiente. Los archivos corridas.csv, historial.csv, ruta.csv, distancias.csv y resumen.json conservan los valores completos para auditoría.

Comparador / indicador	Distancia o valor
Orden inicial	152.294241 km
Vecino más cercano	73.672114 km
Búsqueda aleatoria: promedio de sus 30 mejores	87.440157 km
AG referencia: mejor de 30	69.706205 km
AG referencia: promedio de 30	71.193627 km
AG referencia: desviación estándar muestral	1.013630 km
AG referencia: peor de 30	73.342114 km
Óptimo exacto	69.706205 km
Corridas que alcanzan el óptimo	6 de 30 (20%)
Brecha de la mejor / brecha del promedio	0.000000% / 2.133846%
Semilla de la mejor corrida	10

Mejor ruta: 0 → 11 → 6 → 3 → 8 → 4 → 10 → 7 → 2 → 12 → 5 → 1 → 9 → 0. Longitud: 69.706205 km. La ruta recorre todos los clientes exactamente una vez y coincide en costo con el resultado exacto; en una matriz simétrica, la secuencia inversa tiene el mismo costo.

La mejor solución reduce 54.229258% la distancia del orden inicial y 5.383190% la del vecino más cercano. La primera comparación usa una línea base deliberadamente simple; la segunda muestra que la ventaja es menor frente a una heurística razonable. No se concluye que un AG siempre mejore al vecino más cercano, ni que estos porcentajes se transfieran a datos reales.

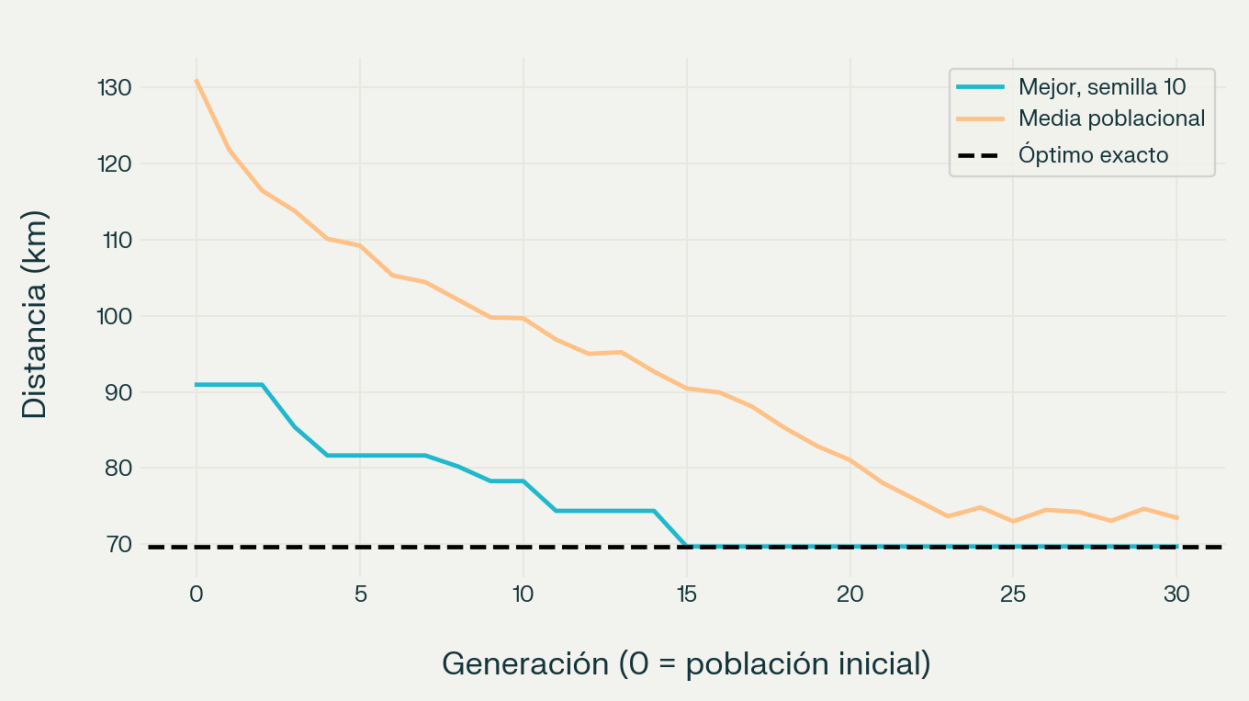


Figura 1. Convergencia de la mejor corrida, semilla 10; backend de referencia ejecutado. La línea discontinua marca el óptimo exacto. Fuente: elaboración propia con la salida del programa.

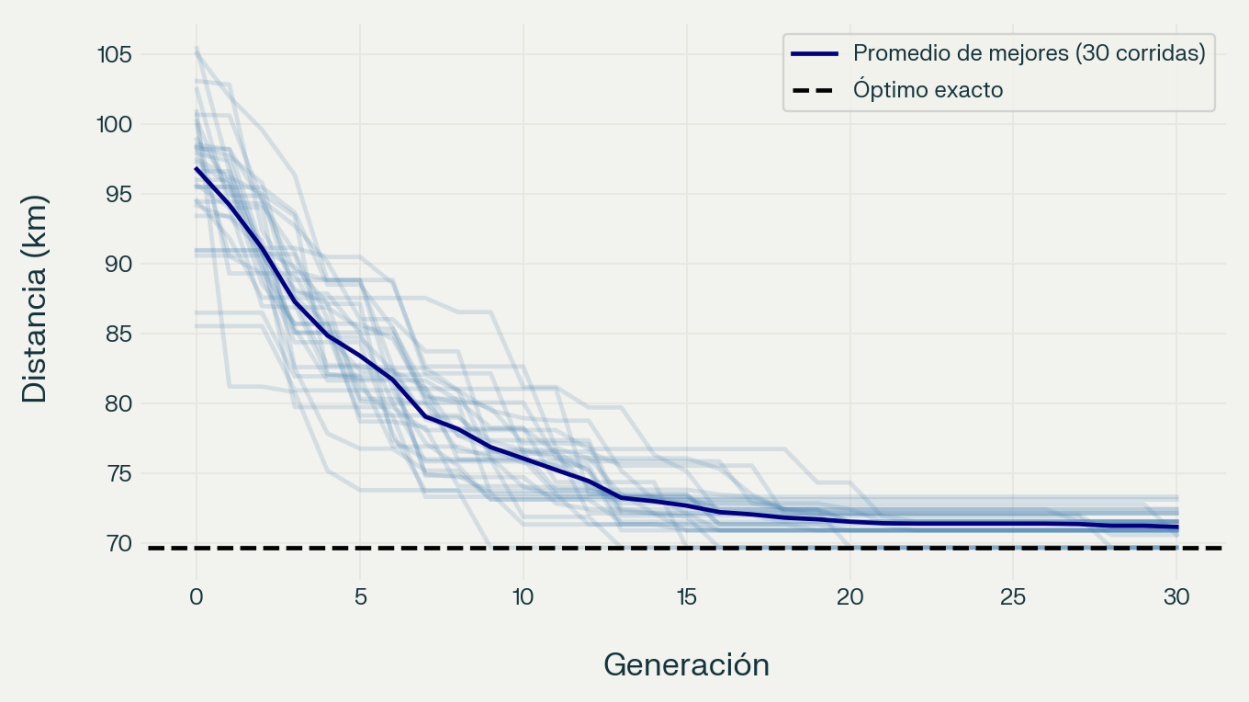


Figura 2. Mejor distancia por generación en las 30 corridas y su promedio. Backend de referencia ejecutado. Fuente: elaboración propia.

IA: Perplexity | Prompt: actividad 2.6 + instrucciones adjuntas | Uso: código y redacción | Verificación: reference; DEAP pendiente.

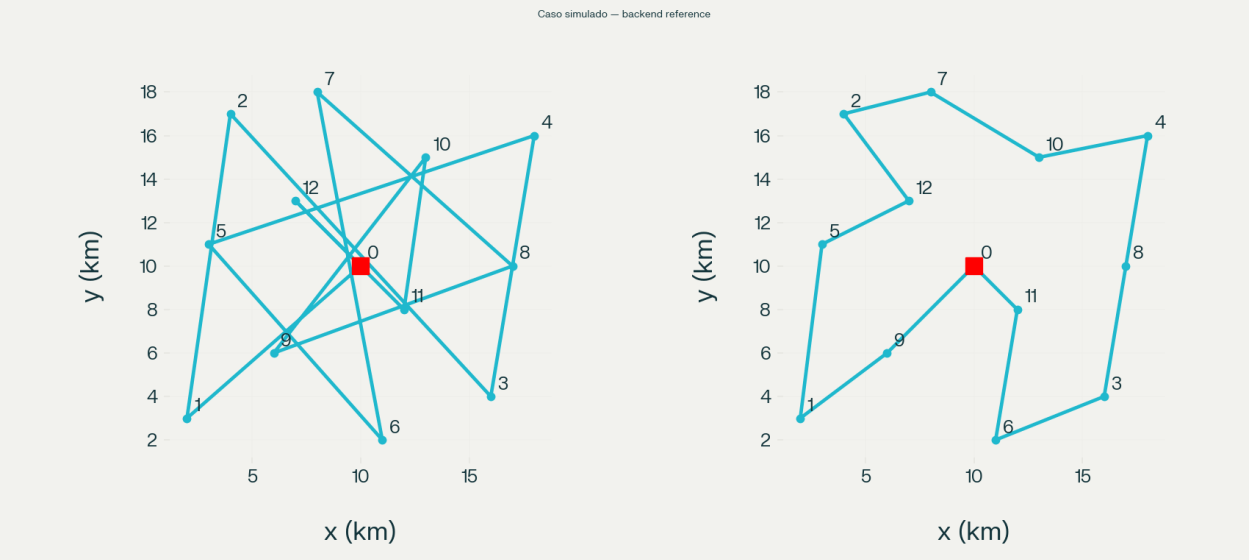


Figura 3. Orden de captura frente a la mejor ruta. Nodos en coordenadas sintéticas, no sobre un mapa vial. Backend de referencia ejecutado. Fuente: elaboración propia.

Interpretación: la población inicial contiene rutas largas; la selección y la recombinación concentran soluciones más cortas. El elitismo preserva la mejor solución, pero no garantiza encontrar el óptimo en 30 generaciones. Que solo 6 de las 30 corridas lo alcancen demuestra variabilidad y evita presentar una única ejecución favorable como garantía general.

9. Validación y límites de la evidencia

Prueba / control	Estado observado
Costo de ruta cerrada con triángulo 3–4–5	Aprobado: 12 km.
Óptimo exacto de un cuadrado unitario	Aprobado: 4 km.
Rechazo de permutación repetida	Aprobado.
Reproducibilidad de reference, semilla 7	Aprobado: ruta e historial idénticos.
Prueba automatizada de DEAP	Omitida: biblioteca no instalada.
Controles internos en las 30 corridas	Sin fallos: validez, elitismo y 31 registros.

Registro de unittest: cinco pruebas detectadas, cuatro aprobadas y una omitida. “OK (skipped=1)” no equivale a validar DEAP. En la máquina del estudiante debe desaparecer la omisión y la prueba test_deap debe terminar en ok. No se realizaron ensayos de campo ni se validó la instalación de las versiones objetivo de requirements.txt en este entorno.

Limitaciones del modelo: geometría euclidiana, un solo vehículo y ausencia de restricciones de operación. Limitaciones del experimento: una sola instancia pequeña, parámetros fijos y sin inferencia estadística formal. La certificación del óptimo pertenece únicamente a estos datos, no a todos los problemas TSP. El programa limita a 16 clientes la verificación exacta; ampliar ese umbral sin analizar memoria y tiempo no es una mejora automática.

10. Instructivo de replicación paso a paso

1. Extraer el paquete, abrir la carpeta Proyecto_2_6 y comprobar que proyecto_ag.py, test_proyecto.py, requirements.txt y clientes.csv estén juntos.

2. Instalar Python 3.12 y abrir una terminal. Crear el entorno con: python -m venv .venv.

3. Activar en Windows: .venv\Scripts\activate. En macOS/Linux: source .venv/bin/activate.

4. Instalar dependencias: python -m pip install -r requirements.txt. El archivo fija DEAP 1.4.3 y Matplotlib 3.10.7 como versiones objetivo; verificar la instalación local.

5. Ejecutar pruebas: python -m unittest -v test_proyecto.py. Deben pasar las cinco pruebas; no aceptar una omisión de test_deap como verificación completa.

6. Ejecutar el proyecto requerido: python proyecto_ag.py --backend deap --out resultados_deap. El programa procesa las 30 semillas y calcula los controles exactos.

7. Abrir resultados_deap/resumen.json y comprobar backend="deap", generaciones=30, corridas=30 y versión de biblioteca. Revisar 930 filas de datos en historial.csv (30 × 31, más encabezado), 30 filas en corridas.csv y 13 tramos en ruta.csv.

8. Revisar gráficas y comparar la suma de los tramos con best_km. Verificar brechas no negativas salvo error numérico diminuto; no exigir que todas las corridas alcancen el óptimo.

9. Sustituir las cifras, figuras y etiquetas de la sección 8 por la salida DEAP real, y actualizar el estado de validación en portada, resumen, sección 9 y conclusiones. Conservar identificada la salida reference si se usa como comparación adicional.

10. Para reproducir solo la evidencia incluida: python proyecto_ag.py --backend reference --out resultados_reference_nuevo. No usar esa carpeta como evidencia de DEAP.

11. Completar nombres de todos los integrantes, matrículas, institución, docente y grupo. Revisar formato del Word y exportar un nuevo PDF individual de la actividad 2.6.

12. Conservar el CSV original para comparar; si se cambian las coordenadas, repetir todo el experimento. Usar ID consecutivos desde 0 y evitar coordenadas no finitas.

11. Entrega y publicación en GitHub

Las instrucciones generales exigen una cuenta GitHub con el nombre del alumno y una carpeta pública del curso. Esta actividad corresponde a las últimas secciones y debe reconocer el trabajo en equipo. Completar en la portada a todos los participantes y acordar quién mantiene el repositorio, sin omitir las reglas específicas del docente sobre cuentas individuales.

Procedimiento: crear la cuenta y un repositorio público; crear una carpeta del curso y la subcarpeta actividad_2_6; subir el .py, pruebas, CSV, requirements, README, Word y PDF actualizado. Incluir resultados_deap para trazabilidad. No subir el entorno .venv ni cachés. Abrir el enlace en una ventana sin sesión para verificar acceso. Enlace final del repositorio: [Completar después de publicarlo].

La cuenta y el repositorio no fueron creados ni publicados por el asistente. El documento separado de complicaciones es material personal de comprensión y no se incorpora al reporte académico.

12. Conclusiones

La solución implementada representa adecuadamente una ruta como permutación y separa datos, evaluación, operadores, experimentación y exportación. El código principal usa DEAP y permite repetir 30 generaciones con 30 semillas. El diseño incluye comparadores más exigentes que el orden de captura y una certificación exacta de la instancia pequeña.

IA: Perplexity | Prompt: actividad 2.6 + instrucciones adjuntas | Uso: código y redacción | Verificación: reference; DEAP pendiente.

La ejecución de referencia obtuvo 69.706205 km como mejor resultado, igual al óptimo exacto; el promedio fue 71.193627 km y el 20% de las corridas alcanzó el óptimo. La comparación con vecino más cercano acota el alcance de la ventaja y la variabilidad confirma que el AG no garantiza éxito en todas las corridas.

No se afirma que DEAP haya sido ejecutado en el entorno actual. La conclusión definitiva sobre la versión exigida requiere instalar la biblioteca, aprobar su prueba y reemplazar los resultados preliminares por los archivos generados con --backend deap. Una aplicación real deberá reemplazar coordenadas sintéticas por distancias viales y añadir capacidad, tiempos y condiciones de servicio, reformulando el problema si es necesario.

13. Referencias (APA, 7.ª edición)

Fortin, F.-A., De Rainville, F.-M., Gardner, M.-A., Parizeau, M., & Gagné, C. (2012). DEAP: Evolutionary algorithms made easy. Journal of Machine Learning Research, 13, 2171–2175. https://jmlr.org/papers/v13/fortin12a.html

DEAP Project. (s. f.-a). Evolutionary tools. DEAP 1.4.3 documentation. Recuperado el 26 de octubre de 2026, de https://deap.readthedocs.io/en/master/api/tools.html

DEAP Project. (s. f.-b). Operators and algorithms. DEAP 1.4.3 documentation. Recuperado el 26 de octubre de 2026, de https://deap.readthedocs.io/en/master/tutorials/basic/part2.html

cstheory.stackexchange.com. (s. f.). Time complexity of Bellman-Held-Karp algorithm for TSP, take 2 [Pregunta y respuestas de foro]. https://cstheory.stackexchange.com/questions/3666/time-complexity-of-bellman-held-karp-algorithm-for-tsp-take-2

Nota bibliográfica: la consigna menciona a Wirsansky, capítulos 3–5, pero no identifica edición ni año. No se atribuyen lecturas de capítulos no proporcionados; incorporar la referencia completa de la edición utilizada en clase si el equipo la consulta. Las fuentes efectivamente utilizadas se listan arriba.

14. Declaración de uso de inteligencia artificial

Herramienta: Perplexity, asistente de IA. Fecha de elaboración: 26 de octubre de 2026. Prompt del usuario: “ayudame con la actividad 2.6 es el proyecto final asi que quiero que se vea robusto quiero que sigas las indicacionesde lo que debe implementar y quiero en un resumen las complicaciones que tuvo el proyecto a la hora del desarrollo, este resumen quiero que no lo agregues al reporte de la actividad sino para que yo entienda el proyecto de mejor manera quiero que sea en word y si te pide otro formato tambien me lo compartes sigue las intrtucciones generales”. Se adjuntaron imágenes con la consigna y las instrucciones generales.

Uso: apoyo en planteamiento del caso simulado, formulación, programación, preparación de pruebas, consulta de documentación, análisis, gráficas y redacción del reporte. Verificación realizada: ejecución del backend de referencia, comprobación del costo exacto y cuatro pruebas automatizadas aprobadas. Verificación pendiente: instalación y ejecución DEAP, quinta prueba y revisión personal por el equipo. El alumnado debe comprender y revisar cada resultado; no declarar como propia una comprobación aún no realizada.

Anexo A. Código Python completo

Archivo ejecutable: proyecto_ag.py. La opción predeterminada es DEAP. Se reproduce completo para satisfacer el requisito de incluir código en el PDF; el archivo .py adjunto evita errores por copiar desde un documento.

```
"""Actividad 2.6: TSP empresarial simulado. Backend principal: DEAP.
El backend reference es independiente y no certifica la ejecución de DEAP.
"""
import argparse, csv, json, math, random, statistics, sys, platform
from pathlib import Path
from functools import lru_cache
from copy import deepcopy
import matplotlib
matplotlib.use('Agg')
import matplotlib.pyplot as plt



def read_data(path):
    with open(path, encoding='utf-8') as f:
        rows = list(csv.DictReader(f))
    if [int(r['id']) for r in rows] != list(range(len(rows))):
        raise ValueError('Los ID deben ser consecutivos desde 0 (depósito).')
    if not 3 <= len(rows) <= 17:
        raise ValueError('Se admiten de 2 a 16 clientes para la verificación exacta.')
    points = [(float(r['x_km']), float(r['y_km'])) for r in rows]
    if not all(math.isfinite(v) for p in points for v in p):
        raise ValueError('Coordenadas no finitas.')
    return points



def matrix(points):
    return [[math.hypot(x-u,y-v) for u,v in points] for x,y in points]



def distance(ind, d):
    route = [0] + [x+1 for x in ind] + [0]
    return sum(d[a][b] for a,b in zip(route,route[1:]))



def valid(ind, n):
    return len(ind)==n and sorted(ind)==list(range(n))



def ordered(a,b):
    # Implementación de referencia de OX; no importa ni sustituye DEAP.
    n=len(a); lo,hi=sorted(random.sample(range(n),2))
    def child(p,q):
        out=[None]*n; out[lo:hi+1]=q[lo:hi+1]
        remaining=[p[(hi+1+i)%n] for i in range(n)
                   if p[(hi+1+i)%n] not in q[lo:hi+1]]
        slots=[(hi+1+i)%n for i in range(n) if not lo <= (hi+1+i)%n <= hi]
        for pos,val in zip(slots,remaining): out[pos]=val
        return out
    x,y=child(a,b),child(b,a); a[:]=x; b[:]=y
    return a,b



def shuffle(ind, indpb):
    for i in range(len(ind)):
        if random.random()<indpb:
            j=random.randint(0,len(ind)-2)
            if j>=i: j+=1
            ind[i],ind[j]=ind[j],ind[i]
    return ind,



def exact(d):
    n=len(d)-1
    @lru_cache(None)
    def cost(mask,j):
        previous=mask^(1<<j)
        if not previous: return d[0][j+1]
        return min(cost(previous,k)+d[k+1][j+1]
                   for k in range(n) if previous&(1<<k))
```

```
    mask=(1<<n)-1
    end=min(range(n),key=lambda j:cost(mask,j)+d[j+1][0])
    optimum=cost(mask,end)+d[end+1][0]
    rev=[]
    while mask:
        rev.append(end); previous=mask^(1<<end)
        if not previous: break
        end=min((k for k in range(n) if previous&(1<<k)),
                key=lambda k:cost(previous,k)+d[k+1][end+1])
        mask=previous
    route=rev[::-1]
    assert valid(route,n) and math.isclose(distance(route,d),optimum,rel_tol=1e-12)
    return optimum,route



def nearest(d):
    remaining=set(range(1,len(d))); route=[]; last=0
    while remaining:
        nxt=min(remaining,key=lambda j:(d[last][j],j))
        remaining.remove(nxt); route.append(nxt-1); last=nxt
    return route



def run_ga(d, seed, backend='deap', popsize=200, generations=30,
           cxpb=.85, mutpb=.25, indpb=.08, elite=2):
    random.seed(seed); n=len(d)-1
    if backend=='deap':
        from deap import base, creator, tools
        if not hasattr(creator,'FitnessMin26'):
            creator.create('FitnessMin26',base.Fitness,weights=(-1.0,))
            creator.create('Route26',list,fitness=creator.FitnessMin26)
        tb=base.Toolbox()
        tb.register('indices',random.sample,range(n),n)
        tb.register('individual',tools.initIterate,creator.Route26,tb.indices)
        tb.register('population',tools.initRepeat,list,tb.individual)
        tb.register('evaluate',lambda x:(distance(x,d),))
        tb.register('select',tools.selTournament,tournsize=3)
        tb.register('mate',tools.cxOrdered)
        tb.register('mutate',tools.mutShuffleIndexes,indpb=indpb)
        pop=tb.population(n=popsize)
        for x in pop: x.fitness.values=tb.evaluate(x)
        mate,mutate=tb.mate,tb.mutate
    else:
        pop=[random.sample(range(n),n) for _ in range(popsize)]
        mate=ordered; mutate=lambda x:shuffle(x,indpb)
    history=[]
    for gen in range(generations+1):
        values=[distance(x,d) for x in pop]
        history.append(dict(seed=seed,generation=gen,best_km=min(values),
                            mean_km=statistics.mean(values),
                            diversity=len({tuple(x) for x in pop})/popsize))
        if gen==generations: break
        elites=[deepcopy(x) for x in sorted(pop,key=lambda x:distance(x,d))[:elite]]
        if backend=='deap':
            children=[tb.clone(x) for x in tb.select(pop,popsize-elite)]
        else:
            children=[deepcopy(min([random.choice(pop) for _ in range(3)],
                      key=lambda x:distance(x,d))) for _ in range(popsize-elite)]
        for a,b in zip(children[::2],children[1::2]):
            if random.random()<cxpb: mate(a,b)
        for x in children:
            if random.random()<mutpb: mutate(x)
            assert valid(x,n), 'Permutación inválida'
            if backend=='deap':
                del x.fitness.values
                x.fitness.values=tb.evaluate(x)
        pop=elites+children
    assert all(history[i+1]['best_km']<=history[i]['best_km']+1e-10
               for i in range(generations))
    assert len(history)==31 and len(pop)==popsize
    best=min(pop,key=lambda x:distance(x,d))
    return list(best),history



def write_csv(path, rows):
    with open(path,'w',newline='',encoding='utf-8') as f:
        w=csv.DictWriter(f,fieldnames=list(rows[0])); w.writeheader(); w.writerows(rows)
```

```
def main():
    parser=argparse.ArgumentParser()
    parser.add_argument('--backend',choices=['deap','reference'],default='deap')
    parser.add_argument('--data',default=str(Path(__file__).with_name('clientes.csv')))
    parser.add_argument('--out',default='resultados_deap')
    args=parser.parse_args()
    if args.backend=='deap':
        try: import deap
        except ImportError:
            raise SystemExit('Falta DEAP. Instala: python -m pip install -r requirements.txt')
    out=Path(args.out); out.mkdir(parents=True,exist_ok=True)
    points=read_data(args.data); d=matrix(points); n=len(points)-1
    assert all(abs(d[i][j]-d[j][i])<1e-12 for i in range(n+1) for j in range(n+1))
    optimum,opt_route=exact(d); nn=nearest(d); original=list(range(n))
    histories=[]; runs=[]; routes=[]
    for seed in range(30):
        route,h=run_ga(d,seed,args.backend); length=distance(route,d)
        histories+=h; routes.append(route)
        # Comparador aleatorio con igual cantidad de candidatos puntuados.
        rng=random.Random(10000+seed)
        budget=200+30*198
        random_best=min(distance(rng.sample(range(n),n),d) for _ in range(budget))
        runs.append(dict(seed=seed,best_km=length,gap_pct=100*(length/optimum-1),
                         random_best_km=random_best))
    chosen=min(range(30),key=lambda i:runs[i]['best_km'])
    best=routes[chosen]; lengths=[r['best_km'] for r in runs]
    meta=dict(backend=args.backend,python=platform.python_version(),
              deap_version=getattr(sys.modules.get('deap'),'__version__',None),
              points=len(points),customers=n,population=200,generations=30,
              runs=30,seeds=list(range(30)),cxpb=.85,mutpb=.25,indpb=.08,
              tournament=3,elite=2,best_seed=chosen,best_km=min(lengths),
              mean_km=statistics.mean(lengths),std_km=statistics.stdev(lengths),
              worst_km=max(lengths),exact_km=optimum,
              hits=sum(math.isclose(x,optimum,abs_tol=1e-8) for x in lengths),
              original_km=distance(original,d),nearest_km=distance(nn,d),
              random_mean_km=statistics.mean(r['random_best_km'] for r in runs),
              route=[0]+[i+1 for i in best]+[0],
              exact_route=[0]+[i+1 for i in opt_route]+[0],
              validation='Permutaciones, matriz, elitismo, 31 registros y costo exacto: OK')
    (out/'resumen.json').write_text(json.dumps(meta,indent=2,ensure_ascii=False),encoding='utf-8')
    write_csv(out/'corridas.csv',runs); write_csv(out/'historial.csv',histories)
    full=[0]+[x+1 for x in best]+[0]; acc=0; legs=[]
    for k,(a,b) in enumerate(zip(full,full[1:]),1):
        acc+=d[a][b]; legs.append(dict(tramo=k,origen=a,destino=b,km=d[a][b],acumulado_km=acc))
    write_csv(out/'ruta.csv',legs)
    write_csv(out/'distancias.csv',[dict(origen=i,**{f'nodo_{j}':d[i][j] for j in range(n+1)}) for i in range(n+1)])
    plt.figure(figsize=(8,4.5))
    selected=[r for r in histories if r['seed']==chosen]
    plt.plot(range(31),[r['best_km'] for r in selected],label=f'Mejor, semilla {chosen}')
    plt.plot(range(31),[r['mean_km'] for r in selected],label='Media poblacional')
    plt.axhline(optimum,color='black',ls='--',label='Óptimo exacto')
    plt.xlabel('Generación (0 = población inicial)'); plt.ylabel('Distancia (km)')
    plt.title(f'Convergencia — backend {args.backend}'); plt.legend(); plt.tight_layout()
    plt.savefig(out/'convergencia.png',dpi=170); plt.close()
    plt.figure(figsize=(8,4.5))
    for seed in range(30):
        plt.plot(range(31),[r['best_km'] for r in histories if r['seed']==seed],alpha=.18,color='steelblue')
    plt.plot(range(31),[statistics.mean(r['best_km'] for r in histories if r['generation']==g) for g in
range(31)],color='navy',label='Promedio de mejores (30 corridas)')
    plt.axhline(optimum,color='black',ls='--',label='Óptimo exacto')
    plt.xlabel('Generación'); plt.ylabel('Distancia (km)'); plt.legend(); plt.tight_layout()
    plt.savefig(out/'convergencia_30.png',dpi=170); plt.close()
    fig,axes=plt.subplots(1,2,figsize=(10,4.5))
    for ax,route,title in [(axes[0],[0]+list(range(1,n+1))+[0],'Orden de captura'),(axes[1],full,'Mejor ruta AG')]:
        ax.plot([points[i][0] for i in route],[points[i][1] for i in route],'-o',ms=4)
        for i,(x,y) in enumerate(points): ax.annotate(str(i),(x,y),xytext=(4,4),textcoords='offset points')
        ax.scatter(*points[0],marker='s',s=80,color='red',zorder=5)
        ax.set(title=title,xlabel='x (km)',ylabel='y (km)'); ax.set_aspect('equal'); ax.grid(alpha=.2)
    fig.suptitle(f'Caso simulado — backend {args.backend}'); fig.tight_layout()
    fig.savefig(out/'rutas.png',dpi=170); plt.close(fig)
    print(json.dumps(meta,indent=2,ensure_ascii=False))

if __name__=='__main__': main()
```

Anexo B. Pruebas automatizadas

```
import importlib.util, math, unittest
from pathlib import Path
import proyecto_ag as p

class ProjectTests(unittest.TestCase):
    def test_exact_square(self):
        d=p.matrix([(0,0),(1,0),(1,1),(0,1)])
        optimum,route=p.exact(d)
        self.assertAlmostEqual(optimum,4.0)
        self.assertTrue(p.valid(route,3))
    def test_closed_route(self):
        d=p.matrix([(0,0),(3,0),(0,4)])
        self.assertAlmostEqual(p.distance([0,1],d),12.0)
    def test_bad_permutation(self):
        self.assertFalse(p.valid([0,0,2],3))
    def check_backend(self,backend):
        points=p.read_data(Path(__file__).with_name('clientes.csv'))
        d=p.matrix(points)
        a,h=p.run_ga(d,7,backend); b,j=p.run_ga(d,7,backend)
        self.assertEqual(a,b); self.assertEqual(h,j)
        self.assertEqual(len(h),31)
        self.assertTrue(p.valid(a,12))
        self.assertTrue(all(h[i+1]['best_km']<=h[i]['best_km']+1e-9 for i in range(30)))
    def test_reference(self): self.check_backend('reference')
    @unittest.skipUnless(importlib.util.find_spec('deap'),'DEAP no instalado; validación pendiente')
    def test_deap(self): self.check_backend('deap')

if __name__=='__main__': unittest.main(verbosity=2)
```

Anexo C. Lista de comprobación antes de entregar

[ ] Completar portada y todos los integrantes del equipo.

[ ] Instalar DEAP y aprobar cinco pruebas sin omisiones.

[ ] Ejecutar las 30 corridas DEAP y sustituir resultados preliminares.

[ ] Revisar formulación, ruta, tablas y gráficas.

[ ] Actualizar declaración de uso de IA con la verificación realizada.

[ ] Exportar PDF individual y conservar Word editable.

[ ] Publicar .py y PDF en carpeta pública de GitHub.

[ ] Mantener el resumen de complicaciones fuera del reporte.